1 Evaluation Methodology

This chapter details the methodology used to evaluate the Agentic Project Management (APM) framework. To assess the framework's effectiveness against current industry standards, we conducted a comparative case study involving human participants. The study compares APM, a multi-agent framework that distributes work across specialized agents, against GitHub Spec-kit, a single-agent toolkit that guides one AI agent through a structured specification-driven workflow. Both frameworks are described in Section 1.2.

The evaluation measures three distinct dimensions:

- Economic Efficiency (share of the Copilot monthly request allowance consumed per session)
- Process Efficiency (human cognitive load and interaction burden)
- Functional Efficacy (ability to complete non-trivial tasks)

The remainder of this chapter is organized as follows. Section 1.2 describes the two frameworks under evaluation. The subsequent sections specify the testbed and task, the study design, and the metrics and data collection approach. Section 2 presents the results of the comparison, and Section 3 discusses their implications.

1.1 Research Objectives

The primary objective of this research is to determine whether the structural overhead introduced by APM (i.e., the Planning and Implementation phases) yields a tangible return on investment compared to a single-agent workflow. Prior empirical work on AI coding assistants has concentrated on productivity and task-completion time [2, 6–9], with comparatively little systematic treatment of cost under the subscription-based procurement model through which Copilot is predominantly adopted. This evaluation contributes a cost perspective alongside the process and functional dimensions.

We define the following Research Questions (RQs):

- **RQ1 (Economic Efficiency):**
  - Under GitHub Copilot's subscription-plus-quota pricing structure, does APM's multi-agent coordination consume fewer premium requests per session than Spec-kit's single-agent workflow?
  - If a difference in request consumption is observed, is it large enough to meaningfully shift the share of a buyer's monthly allowance burned per task completion?
  - Where in each framework's workflow (planning phase versus implementation phase) does request consumption concentrate, and does that concentration pattern differ between frameworks?
- **RQ2 (Process Efficiency):**
  - Does the structured coordination of APM reduce the amount of manual effort the user must invest during execution, particularly interventions, corrections, and clarifications?
  - If a difference in user effort is observed, is it significant enough to justify the structural overhead APM introduces?
  - Given that the User remains in the loop as a message carrier, can the agents coordinate effectively enough that the User's role stays limited to shuttling messages rather than correcting or redirecting agent behavior?
- **RQ3 (Functional Efficacy):**
  - Can APM successfully deliver complex, non-trivial features in scenarios where single-agent workflows are prone to failure due to context exhaustion, loss of coherence across long sessions, or inability to coordinate work across multiple concerns?
  - How does the performance of APM compare to the performance of a single-agent workflow?

1.2 Frameworks Under Evaluation

The two frameworks are both open-source and spec-driven, running on the same AI IDE platform. Both structure work through explicit specification and planning artifacts before implementation begins. They differ in how execution is organized: Spec-kit operates within a single agent conversation, while APM distributes work across multiple specialized agents.

1.2.1 GitHub Spec-kit

Spec-kit[1] is a specification-driven development toolkit developed by GitHub. It guides a single AI agent through a structured workflow using three sequential commands:

1. `/speckit.specify`: transforms a feature description into a structured specification containing requirements, user stories, and acceptance criteria.
2. `/speckit.plan`: converts the specification into a technical implementation plan with architecture decisions, data models, API contracts, and supporting documents.
3. `/speckit.tasks`: derives an executable task list (`tasks.md`) from the plan, with parallelism annotations and sequencing constraints.

Execution is triggered by a fourth command, `/speckit.implement`, which parses the task list and carries out each task in sequence. Additional commands support iterative refinement (`/speckit.clarify`), cross-artifact consistency checking (`/speckit.analyze`), and custom quality checklists (`/speckit.checklist`). A constitution mechanism, maintained via the `/speckit.constitution` command, allows users to establish architectural rules that the agent enforces throughout the workflow.

All planning and implementation happens within one continuous conversation. The agent acts as both planner and executor, and the full specification, plan, and task context accumulates in a single context window. Spec-kit produces a fixed set of artifact types (specification, plan, data model, contracts, research, tasks) regardless of project type; the templates are predefined and every project receives the same artifact layout.

1.2.2 APM in Comparison

Where Spec-kit runs the entire workflow in a single conversation, APM (described in detail in the preceding chapters) separates planning, coordination, and execution across three agent roles, each operating in its own conversation with deliberately scoped context. Agents communicate through a file-based Message Bus rather than sharing a context window, planning documents adapt their structure to the project rather than matching to a predetermined artifact layout, and the user acts as a message carrier between conversations. When an agent's context fills, an explicit Handoff mechanism preserves working knowledge across context boundaries.

[1] Documentation available at: https://github.github.com/spec-kit/

The evaluation tests whether this multi-agent overhead produces measurably better outcomes than Spec-kit's single-agent approach. The task is designed to be complex enough to challenge a single context window, making it a meaningful test of whether distributing work across specialized agents with scoped context yields tangible benefits in practice.

1.3 Testbed and Task Design

To evaluate the frameworks under realistic constraints, we utilized an existing open-source project, the `eclass-mcp-server`,[2] a Python-based Model Context Protocol (MCP)[3] server designed to interface with the `openeclass` platform.[4] Openeclass is an online platform for Greek universities that allows students to manage their courses and assignments.

1.3.1 Task Description

The task requires participants to implement two new announcement tools in the Python MCP server and to create a complete native C replica of the entire server.[5] The two tools are:

- `get_course_announcements`: retrieves announcements for a specific course.
- `get_general_announcements`: aggregates system-wide and per-course announcements.

The C implementation adds significant complexity: participants must manage code translation, memory safety, and protocol details across two very different languages. This combination of feature implementation and full-language replication is calibrated to stress-test the limits of agent coordination and expose any differences between the two frameworks, especially when context management becomes a bottleneck.

Key technical challenges:

1. Dynamic Data Source: the required data for implementing the new features is not available in static HTML for parsing. Agents must research the legacy codebase to discover a specific AJAX endpoint and its behavior.
2. Nested Parsing: the server returns JSON where the payload is an array of raw HTML strings, requiring a second layer of parsing to extract the data.
3. Client-Side Filtering: the legacy API does not support date filtering. The agent must implement robust client-side logic to parse and filter unique locale-formatted dates.
4. C Implementation: agents must build a native C MCP server replica, manually implementing JSON-RPC 2.0 with standard C string manipulation (no external JSON libraries such as cJSON/Jansson). Use of `libcurl` for HTTP/HTTPS and `libxml2` for HTML parsing was required. A working Makefile must build the C binary.

[2] Repository available at: https://github.com/sdi2200262/eclass-mcp-server
[3] Protocol specification available at: https://modelcontextprotocol.io
[4] Repository available at: https://github.com/gunet/openeclass
[5] Task specification and grading infrastructure available at:
https://github.com/sdi2200262/bsc-apm-case-study-task

1.3.2 Grading

Submissions are evaluated using an automated black-box test suite with comprehensive validation, grading, metric extraction, and dataset aggregation capabilities (see the evaluation appendix for detailed infrastructure documentation).[6]

The test suite awards 100 points total: 12 points for baseline functionality (existing Python tools), 36 points for the two new Python features (18 points each), and 52 points for the complete C replica implementation. A submission with no patch applied receives the baseline score of 12/100 points.

All tests run against fixed baseline commits to ensure reproducibility: `eclass-mcp-server` at commit `8b4e39c` and `openeclass` at commit `e8b3329`.

1.3.3 Repository Architecture and Data Management

The case study employs a two-repository system separating public resources from sensitive participant data. The task repository (`bsc-apm-case-study-task`) is openly accessible and includes task specifications, grading infrastructure, and participant guides. The dataset repository (`bsc-apm-case-study-data`) contains all submitted participant data: code patches, sanitized chat transcripts (see Section 1.3.4), and anonymized session metadata.

Access to the dataset repository requires explicit authorization and compliance with data usage agreements, enabling peer review and reproducibility while maintaining participant privacy.

1.3.4 Sanitization and PII Handling

Chat exports contain sensitive information (unstructured user-input text, file paths, command histories, timestamps). To protect participant privacy while enabling reproducibility, all exports are sanitized using reversible encryption before storage.

Each chat is assigned a unique identifier via SHA256 hashing of concatenated user messages for anonymous cataloging. Essential metadata (chat ID, model IDs, request count) is preserved in plaintext to enable dataset organization without decryption, while conversation content is encrypted using an HMAC-SHA256 XOR cipher. Detailed technical specifications are provided in the evaluation appendix.

[6] Grading infrastructure available at: https://github.com/sdi2200262/bsc-apm-case-study-task/tree/main/grader

1.4 Study Design

The study is conducted on a single AI IDE platform, VS Code with GitHub Copilot, and compares the two frameworks across a homogeneous cohort of participants. It is designed as a within-subjects experiment, so that each participant contributes data points for both frameworks and per-participant variance in skill, prior experience, and working style is absorbed into each pairwise comparison rather than confounding it.

1.4.1 Experimental Design

Each participant completes the case study task twice: once with APM, once with Spec-kit. The order of the two frameworks is counterbalanced across participants, so that approximately half of the cohort encounters APM first and the other half encounters Spec-kit first. This counterbalancing lets us attribute any observed first-to-second session improvement to task familiarity rather than to the framework itself, and is analyzed explicitly in Section 2 under Learning Effects.

The two sessions are isolated from one another using Git, so no code, state, or context from the first session can influence the second. The participation workflow including session isolation and patch submission are documented in the evaluation appendix.

1.4.2 Data Source

The primary data source for the study is GitHub Copilot chat exports, captured in JSON form from the VS Code Command Palette at the end of each session, alongside each participant's patch submission. Copilot's export format provides structured records of every conversation, including model identifiers, message timestamps, tool invocations, and per-message speaker roles. This granularity enables programmatic extraction of both request-level and interaction-level metrics directly from the exports, without manual annotation. Because GitHub Copilot is billed per seat with a monthly premium-request allowance and a marginal per-request charge for overage consumption, the per-conversation request counts preserved in the export metadata correspond directly to the unit in which the buyer's cost is incurred.[7]

APM's multi-agent workflow spans multiple Copilot conversations within a single session (one for the Planner, one for the Manager, and multiple for the Workers), while Spec-kit operates within a single conversation. To keep the two frameworks comparable, all metrics are computed by aggregating every conversation file belonging to a participant submission into a unified session-level view, so that the total interaction cost of the session is captured regardless of how the work was distributed across conversations.

[7] Premium-request billing structure and overage rates documented at: https://docs.github.com/en/billing/concepts/product-billing/github-copilot-premium-requests

1.4.3 Participant Pool

Participants are drawn from a cohort of University students and working professionals with active access to GitHub Copilot, either through the GitHub Education Pack or through a standard premium subscription.

1.5 Metrics and Data Collection

The metrics collected for this study are organized into five categories: session metadata, the three dimensions introduced above (economic efficiency, process efficiency, and functional efficacy), and the functional grading rubric. The subsections below specify each category in turn.

1.5.1 Session Metadata

[Forthcoming. Will cover the distinct model identifiers used across a session, the per-conversation and project-level timestamps, and the session count for each APM submission.]

1.5.2 Cost Metrics

GitHub Copilot is billed as a three-part tariff: a per-seat monthly access price, an included allowance of premium requests per user per month, and a marginal charge of $0.04 for each premium request beyond the allowance.[8] The structure is uniform across the Student plan (300 included premium requests per month), the Pro plan (300), the Pro+ plan (1,500), the Business plan (300 per user), and the Enterprise plan (1,000 per user). Each supported model is associated with a premium-request multiplier that determines how many premium requests a single model interaction deducts from the allowance: a 1× model consumes one premium request per interaction, while advanced reasoning or fast-mode models apply higher multipliers (for example, 3× or 30×). A framework that consumes fewer premium requests to complete the same task therefore consumes a smaller share of the buyer's monthly allowance. Economic research on three-part tariff choice shows that buyer preference for this pricing structure is driven by demand uncertainty rather than marginal price alone, with consumers steering toward allowance-based plans to hedge against bill variance [4, 5].

The primary cost metric for the study is **premium requests per session**: the total number of premium requests consumed across all conversations of a single framework session. Each participant contributes one APM-session count and one Spec-kit-session count for the same task, aggregated and compared across the cohort. This metric maps directly onto the buyer's budget exposure under the subscription plans described above. The metric is deliberately decoupled from the task rubric: request counts are compared between frameworks for the same completed work, without attempting to attribute requests to individual rubric points or features.

The secondary cost metric is a phase-broken breakdown of the per-session request count. For APM, the planning phase corresponds to the Planner conversation and the implementation

[8] Tiered premium-request allowances documented at: https://docs.github.com/en/copilot/get-started/plans

phase to the Manager and Worker conversations, a boundary marked naturally by the conversation identity in the chat export. For Spec-kit, the planning phase covers all requests up to and including the last `/speckit.tasks` invocation, together with any preceding commands in the planning cluster: `/speckit.constitution`, `/speckit.specify`, `/speckit.plan`, `/speckit.clarify`, `/speckit.analyze`, and `/speckit.checklist`. The implementation phase covers all requests from `/speckit.implement` onward. The phase split locates where each framework spends its budget and exposes the source of any observed cost differential between frameworks.

All sessions use a single underlying model, held fixed across both frameworks and across both sessions of each participant: GPT-5.4. This is a 1× premium-request model, available to participants on the Student plan as well as the Pro, Pro+, Business, and Enterprise tiers.[9] Holding the model fixed at a uniform 1× multiplier eliminates multiplier-induced variance in the cost metric, so that a difference in premium-request counts reflects a difference in framework behavior rather than a difference in model choice.

The study does not measure raw inference cost in terms of tokens consumed or dollar-equivalent API pricing. Recent empirical work on the LLM market documents that the raw inference price of a comparably capable model has declined by roughly three orders of magnitude over the preceding two years, and that the same model hosted by competing providers can differ in price by as much as 90% [3]. A subscription buyer is insulated from both the deflationary curve and the cross-provider heterogeneity: the cost paid per month is fixed by the plan, and the cost paid per request is set by the vendor's multiplier table rather than by inference economics. Measuring dollar-equivalent token cost would therefore reflect a quantity that neither framework's user actually incurs. GitHub Actions minutes consumed by Copilot cloud agents are also out of scope: neither APM nor Spec-kit invokes the cloud agent feature in the testbed task, so the dual-consumption mechanism documented for cloud agents does not apply.[10]

1.5.3 Reliability Metrics

The reliability metrics capture how autonomously each framework executes work once a task has been delegated to an AI agent. Prior empirical work has shown that the acceptance rate of AI-generated output is one of the strongest observable predictors of perceived productivity [9], and that acceptance rates in field settings can fall below 44% for experienced developers working on mature codebases [1]. When suggestions are accepted but later modified, the modification itself reflects a reliability gap: the agent produced output close enough to be useful but not correct enough to be used as is. The metrics below operationalize these reliability signals directly from the tool-call and code-modification events in the Copilot chat export.

The **tool success rate** is the proportion of agent tool invocations that return a non-error result. The **tool retry rate** is the proportion of tool invocations followed by a repeated call to the same tool with adjusted arguments within the same agent turn, indicating that the first attempt

[9] Model multiplier values and plan availability documented at: https://docs.github.com/en/copilot/reference/ai-models/supported-models

[10] Dual consumption of premium requests and Actions minutes by Copilot cloud agents documented at: https://docs.github.com/en/billing/concepts/product-billing/github-copilot-premium-requests

failed or was rejected. The **user edit rate on terminal commands** is the proportion of terminal-execution tool calls whose command string was modified by the user before execution rather than executed verbatim, indicating a lack of trust in the agent's generated command. Together these metrics capture the frequency with which agents produce directly usable output rather than output requiring re-attempts or human correction.

1.5.4 Process Metrics

The process-efficiency metrics capture the human coordination burden each framework imposes across a session. Developer productivity is conventionally measured along the SPACE dimensions: satisfaction, performance, activity, communication, and efficiency. Recent empirical work has documented that AI coding assistants shift the allocation of developer time toward review and prompting activities rather than active coding [1, 9]. Becker et al. [1] measured that developers allowed to use AI tools spent 9% of their time reviewing AI output, 8% prompting, and 4% waiting, while accepting fewer than 44% of generated suggestions. The metrics below operationalize the user's coordination effort in a form extractable directly from the Copilot chat exports.

The primary process metrics characterize the distribution and magnitude of user inputs during a session. Each user message is assigned to one of three buckets: short (brief acknowledgments or routing messages), protocol (framework-protocol messages such as APM handoff routing or Spec-kit slash-command invocations), and standard (substantive task-related prompts). The **behavior distribution** reports the share of each bucket in the session's total user-message count. The **behavior magnitude** reports the average character count per bucket. Together these views expose whether a framework's user burden is concentrated in mechanical message-shuttling (high protocol share, low magnitude) or in substantive problem-solving (high standard share, high magnitude).

Session duration is measured as the wall-clock time from the first user message of the session to the last assistant message. For APM, this spans all of the Planner, Manager, and Worker conversations belonging to the session. For Spec-kit, this is the span of the single conversation from session start to completion.

The shape of this metric set follows the verification-burden pattern documented in enterprise-scale AI coding studies [2, 6]. In those settings, developers with heavier daily coding practice extracted disproportionately larger speed benefits from AI assistance, consistent with the hypothesis that verifying generated output imposes a cognitive cost that scales with prompt complexity. A framework that keeps user inputs short and protocol-routed trades substantive prompt engineering for higher-volume mechanical coordination. A framework that pushes the user toward long substantive prompts makes the opposite trade, requiring more per-prompt engineering effort in return for fewer coordinating interactions [8].

1.5.5 Functional Metrics

The functional dimension is evaluated against the 100-point grading rubric introduced in Section 1.3.2. Final code artifacts (`.patch` files) are scored across three dimensions:

- Baseline Regression: ensures that existing functionality (12 points) remains intact after the patch is applied, isolating the framework's impact on code stability.
- Functional Correctness: end-to-end validation of the new Python features (36 points) and the complete C replica (52 points) against the specification.
- Robustness: edge-case handling including authentication failures, invalid inputs, empty responses, and memory safety (Valgrind validation for the C implementation).

This three-dimensional scoring captures partial credit across baseline preservation, feature-level correctness, and robustness, rather than collapsing task completion into a single binary outcome of the kind commonly used in prior empirical work on AI coding assistants [2, 6, 7].

2 Results

2.1 Functional Performance Analysis

Primary metrics: total score distribution, completion rates (100/100, 49–99, 12–48, baseline-only), baseline regression rates, and feature implementation success rates for Python and C.

Category analysis: success rates per test category (build, protocol, login, logout, courses, auth-status, announcements) for both Python and C implementations, identifying framework-specific failure patterns.

Comparative analysis: paired t-tests comparing APM and Spec-kit for total score, category scores, and completion rates. Each test is accompanied by a standardized effect-size estimate, Cohen's d, which expresses the mean difference between two groups in pooled-standard-deviation units and distinguishes statistical significance from practical significance.

[Forthcoming. Quantitative results of the analyses above, including score distributions, per-category success rates, and paired statistical comparisons between APM and Spec-kit with their associated effect-size estimates.]

2.2 Economic and Process Efficiency Analysis

Economic and process metrics defined in Section 1.5 are compared between frameworks on a per-participant paired basis. For the primary cost metric (premium requests per session), paired t-tests compare the APM and Spec-kit distributions across the cohort, with Cohen's d reported alongside each test to quantify effect size. The phase-broken secondary cost metric is reported as the mean share of planning-phase versus implementation-phase requests per framework.

For process metrics, behavior distribution and behavior magnitude are reported by input bucket for each framework. Per-bucket paired t-tests are applied to the participant-level differences across the short, protocol, and standard buckets. Session duration is compared using the same paired t-test approach.

Correlation analysis relates the cost and process metrics to the functional rubric score. Pearson correlation coefficients are reported for premium-requests-per-session against total rubric score, and for session duration against rubric score, both within each framework and across the combined cohort. These correlations surface whether framework-induced variation in cost or process burden translates into functional outcomes.

[Forthcoming. Quantitative results of the analyses above, including per-session request deltas, phase-broken breakdowns, behavior-distribution and magnitude tables, session durations, and the correlation coefficients relating cost and process metrics to functional outcomes.]

2.3 Learning Effects and Confounding

Participants complete the same task twice (Session 1 and Session 2) using different frameworks. This creates a learning effect where task familiarity from S1 influences S2 performance, potentially confounding framework comparisons.

Analysis strategy:

- Learning Effect Matrix: calculate mean improvement from S1 to S2 for both framework orderings across all metrics (functional scores, cost, process efficiency).
- Order Effect: compare APM-first vs. Spec-kit-first directions to test whether order of framework exposure affects learning.
- Baseline Learning: overall mean improvement across both directions (pure task learning effect regardless of framework).

Key questions: does task learning confound framework comparisons? Is there a preferred first framework that maximizes S2 improvement? How should learning effects be controlled for when comparing frameworks?

[Forthcoming. Quantitative results of the learning-effect analysis, including the S1-to-S2 improvement matrix across framework orderings, the order-effect comparison, and the baseline-learning estimate.]

3 Discussion

3.1 Economic Efficiency

[Forthcoming. Synthesises the observed per-session request deltas against Lambrecht, Seim, and Skiera's (2007) allowance-dominance finding and the single-model concentration pattern documented by Demirer et al. (2025), to assess whether measured differences between APM and Spec-kit translate into meaningful cost-exposure shifts for the buyer.]

3.2 Process Efficiency

[Forthcoming. Synthesises the observed behavior-distribution and session-duration deltas against the verification-burden pattern documented by Paradis et al. (2024) and the coordination-time breakdown from Becker et al. (2025), to assess whether APM's structural coordination overhead is offset by a reduced per-interaction verification burden on the user.]

3.3 Functional Efficacy

[Forthcoming. Synthesises rubric scores and completion rates against the Cui et al. (2025) +26% enterprise baseline and the Becker et al. (2025) slowdown finding on mature codebases, to characterise each framework's efficacy on complex tasks that stress single-context-window workflows.]

3.4 Threats to Validity

This subsection enumerates internal, external, and construct validity threats and the mitigations applied.

3.4.1 Internal Validity

Learning effects from the within-subjects design. Each participant completes the same task twice, and task familiarity from the first session can influence performance in the second. Counterbalancing the framework order across participants mitigates the framework-effect confound but does not eliminate the underlying learning curve. The Learning Effects analysis in Section 2 quantifies the mean improvement between sessions across both orderings and is the primary defence against this threat.

Participant experience heterogeneity. Productivity and process-efficiency effects of AI coding assistants vary with developer experience and daily coding volume [6, 7, 9]. The within-subjects design partially absorbs cross-participant variance by pairing each participant with themselves, but residual dispersion may limit the precision of cohort-level estimates.

Self-reported versus observed measures. Subjective developer-reported productivity often diverges from objective time-in-motion measurements, occasionally in the opposite direction [1, 7]. This study reports only instrumented, export-derived metrics and does not combine them

with self-reports, which limits the process-efficiency findings to measurable coordination activity rather than subjective effort or satisfaction.

3.4.2 External Validity

Single-vendor, single-model evaluation. The case study is run exclusively on GitHub Copilot with a fixed 1× multiplier model. Empirical enterprise data shows that most API workloads concentrate on a single model even when multi-homing is available, with more than 90% of volume typically routed to a single chosen model [3]. The design therefore reflects empirical adoption patterns, but the findings do not generalise to settings where frameworks dynamically route between models.

Task complexity and domain specificity. The testbed task is a complex software engineering assignment requiring two-language implementation and legacy-codebase research. In an empirical analysis of twelve commercial LLM use-case categories, Demirer et al. [3] report that programming commands both the highest average intelligence index and the highest willingness to pay among buyers. The chosen testbed is therefore representative of the domain most sensitive to marginal AI capability, but not of administrative, analytical, or creative workflows.

Lab-to-field generalisation gap. Lab-style productivity measurements for AI coding assistants have exceeded enterprise field measurements by more than a factor of two, and experienced developers on mature codebases have been observed to be slower with AI than without [1, 2, 7]. The case study is positioned between lab and field conditions, and its measurements are neither pure laboratory benchmarks nor natural-setting observations.

Temporal decay of findings. The AI coding assistant field is evolving rapidly. Specific effect sizes are tied to the model version, the Copilot pricing parameters, and the framework releases available during the participant window. The three-part tariff allowance sizes and model multipliers are subject to vendor change, and cost-efficiency findings are defensible against current pricing but may require re-estimation under materially different parameters [4].

3.4.3 Construct Validity

Cost measured within a single procurement model. The cost metric measures share of the monthly premium-request allowance consumed within the subscription-plus-quota procurement model. It does not measure dollar-equivalent inference cost, which is decoupled from the subscription price and has declined by roughly three orders of magnitude over the preceding two years [3]. The subscription buyer is by definition insulated from that translation, so the scoping is intentional rather than a defect.

Behavioral input buckets as process-efficiency proxy. The short, protocol, and standard input buckets are defined for this study and extracted from the chat-export content. They are not validated against an external coordination-burden instrument such as the SPACE framework [9] or the NASA Task Load Index. Their validity as proxies for human coordination effort rests on face validity alone: the relative ordering of frameworks along these metrics is robust, but the absolute magnitude of differences is interpretive rather than calibrated.

Binary functional outcomes. The 100-point rubric captures correctness and robustness but not code quality, maintainability, security posture, or long-term technical debt. Prior empirical work on AI coding assistants has consistently noted that code-quality dimensions are not captured by pass/fail functional measurement [2, 6], and this study inherits that limitation.

Chat-export data scope. Metric extraction is limited to the per-conversation JSON exports produced from the VS Code Command Palette. Inline code-completion events that do not appear in the chat export are invisible to the metric set. The reliability and process metrics therefore characterise only the agent-driven portion of the workflow rather than the full IDE interaction.

References

[1] Joel Becker, Nate Rush, Beth Barnes, and David Rein. Measuring the impact of early-2025 AI on experienced open-source developer productivity, 2025.

[2] Kevin Zheyuan Cui, Mert Demirer, Sonia Jaffe, Leon Musolff, Sida Peng, and Tobias Salz. The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers. Working paper, February 2025.

[3] Mert Demirer, Andrey Fradkin, Nadav Tadelis, and Sida Peng. The emerging market for intelligence: Pricing, supply, and demand for LLMs. Working Paper 34608, National Bureau of Economic Research, 2025.

[4] Anja Lambrecht, Katja Seim, and Bernd Skiera. Does uncertainty matter? Consumer behavior under three-part tariffs. *Marketing Science*, 26(5):698–710, 2007. doi: 10.1287/mksc.1070.0283.

[5] Bo Li and Subodha Kumar. Managing software-as-a-service: Pricing and operations. *Production and Operations Management*, 31(6):2588–2608, 2022. doi: 10.1111/poms.13729.

[6] Elise Paradis, Kate Grey, Quinn Madison, Daye Nam, Andrew Macvean, Vahid Meimand, Nan Zhang, Ben Ferrari-Church, and Satish Chandra. How much does AI impact development speed? An enterprise-based randomized controlled trial, 2024.

[7] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. The impact of AI on developer productivity: Evidence from GitHub Copilot, 2023.

[8] Maja Vukovic, Rangeet Pan, Tin Kam Ho, Rahul Krishna, Raju Pavuluri, and Michele Merler. Usage, effects and requirements for AI coding assistants in the enterprise: An empirical study. In *Proceedings of the International Conference on Software Engineering (ICSE)*, 2026. arXiv preprint arXiv:2601.20112v1.

[9] Albert Ziegler, Eirini Kalliamvakou, X. Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. Measuring GitHub Copilot's impact on productivity. *Communications of the ACM*, 67(3):54–63, 2024. doi: 10.1145/3633453.